

控制权、进程、线程与调度

沿进程生命周期理解 408 核心机制

408 操作系统专题手册

核心问题

用户程序怎样高速直接运行，而操作系统又始终能够重新取得控制权，并把有限 CPU 时间安全地分给多个执行流？

本册沿用整个 OS 知识体系的统一推演引擎：

State → Transition → Failure (Bad State) → Property (Safety/Liveness)
→ Minimal Mechanism → Tradeoff

对本专题而言，最常见的状态转移可以继续写成：

当前状态 + 事件 $\xrightarrow[\text{策略}]{\text{机制}}$ 新状态

真正要检查的不是“背出了哪个状态图”，而是：这次事件改变了什么状态？什么坏状态必须被排除？内核用什么最小机制重新取得控制并维持安全性与进展性？

本册 Owner / Stop Boundary

本册只完整拥有：task、执行上下文、进程/线程状态、run/wait queue、CPU 调度、block/wakeup 与控制权交接。

同步协议、PV/管程与死锁由 OS-03 拥有；页表与换页由 OS-04 拥有；OFD、dentry/inode 与文件生命周期由 OS-06/07 拥有；设备请求与 DMA 完成路径由 OS-05 拥有。本册在 fork/read/exec 中只保留与进程状态或资源引用交接直接相关的接口，不在这里重讲后续 Topic 的内部机制。

目录

1 核心模型：控制权与资源状态	2
1.1 五个检查维度	2
1.2 两个不能混淆的“状态”	2
2 第零章：为什么内核必须先解决“控制权”问题	2
2.1 受限直接执行：又要快，又不能失控	2
2.2 用户态与内核态：不是“两个程序”，而是权限边界	3
2.3 进入内核的三种典型入口	3

2.4	Trap 之后不一定 Context Switch	3
3	程序、进程与线程：把“资源世界”和“执行路径”拆开	4
3.1	程序是静态蓝图，进程是动态执行环境	4
3.2	线程是执行上下文，而不是第二套资源世界	4
3.3	并发与并行	5
3.4	408 视角下的进程/线程对比	5
4	PCB：把“进程是什么”落实到内核可管理的数据结构	5
4.1	PCB 不是进程本体，而是内核管理进程的控制记录	5
4.2	PCB 与线程上下文：教材抽象和现代实现要分层	6
4.3	TCB：把线程自己的执行状态单独拿出来	6
5	进程通信 IPC：隔离之后怎样交换信息	7
5.1	母问题：进程彼此隔离，为什么还能通信？	7
5.2	四类典型 IPC：看“共享什么、等什么”	7
5.3	共享内存为什么“快”，又为什么不等于“简单”	8
6	进程资源接口：本册只追踪“引用怎样随生命周期交接”	8
7	进程生命周期主线	9
7.1	生命周期总览	9
7.2	五态为什么还不够：挂起与七态模型	9
7.3	阶段 A：创建——从父进程产生新执行环境	10
7.4	阶段 B：exec——不是新建进程，而是替换进程映像	10
7.5	阶段 C：Ready——只缺 CPU	10
7.6	阶段 D：Running——用户态和内核态都可能属于“这个进程正在运行”	11
7.7	阶段 E：Blocked——即使给 CPU 也不能继续	11
7.8	阶段 F：Wakeup——Blocked → Ready，不等于马上 Running	11
7.9	阶段 G：Exit / Zombie / Wait	11
8	调度：控制权已经回到内核，下一步给谁？	12
8.1	先分清三级调度：它们选择的不是同一种“下一位”	12
8.2	机制与策略必须拆开	12
8.3	调度评价指标是互相冲突的目标函数	12
8.4	先识别工作负载，再决定“什么叫好”	13
8.5	经典算法应按“偏好谁”来记	13

8.6	HRRN: 不要只背公式, 要看“等待如何改变优先级”	14
8.7	时间片到底解决什么	14
8.8	有调度原因, 不等于“现在这一条指令就能切换”	14
9	上下文切换: 到底切了什么?	15
9.1	抽象定义	15
9.2	同进程线程切换与跨进程切换	15
9.3	动态轨迹: A 切到 B	15
10	系统调用、中断、异常: 控制流为什么会改变	16
10.1	中断响应要拆成两层: 硬件自动入口与软件处理程序	16
10.2	一次系统调用的五阶段模板	16
10.3	时钟中断为什么是抢占式调度的基础	17
10.4	设备中断: 唤醒并不等于直接交 CPU	17
11	综合题一: 从 fork 到 wait 的进程生命周期	17
11.1	完整轨迹	17
12	综合题二: 一次阻塞 read 的“进程侧投影”	18
13	线程实现模型: 考试模型与现代实现边界	19
13.1	用户级线程与内核级线程的区别: 谁看得见、谁调度	19
13.2	M:1、1:1、M:N 的本质	19
14	408 解题检查表: 从概念辨析到状态推演	19
14.1	进程状态题八问	19
14.2	调度计算题九步	20
14.3	系统调用/中断题九问	20
14.4	PCB / TCB / 进程资源接口题八问	20
14.5	IPC 题六问	21
15	知识树与专题接口	21
	参考与工程边界来源	22

1 核心模型：控制权与资源状态

1.1 五个检查维度

面对进程、线程、调度、系统调用和中断，先回答：

1. 谁现在拥有 CPU 控制权？用户程序还是内核？
2. CPU 上真正正在执行的实体是谁？一个进程中的哪条执行流？
3. 这个执行流依附于哪个资源环境？地址空间、文件表、权限、信号等属于谁？
4. 什么事件会迫使执行路径改变？系统调用、异常、中断、阻塞、唤醒、抢占、退出。
5. 当多个执行实体都想运行时，谁来选择下一位？调度机制如何获得机会，调度策略如何作选择？

进程与调度八问

拿到任何进程/线程/调度题，依次问：

1. 当前运行实体是谁？
2. 当前处于用户态还是内核态？
3. 发生了什么事件？
4. 这个事件只是模式切换，还是会导致调度/上下文切换？
5. 当前实体为什么还能继续/为什么必须等待？
6. 若不能继续，它应进入 Ready 还是 Blocked？
7. 哪些资源继续属于它，哪些状态需要保存？
8. 最终谁获得 CPU，代价是什么？

1.2 两个不能混淆的“状态”

本模块同时出现两类状态：

- 体系结构执行状态：PC、通用寄存器、SP、状态寄存器等，决定“这条执行流下一步从哪里继续”。
- 内核管理状态：任务状态、调度信息、地址空间引用、打开文件、权限、等待对象等，决定“OS 怎样管理它”。

进程/线程切换，本质上就是：保存旧执行上下文，选择新的可运行实体，再恢复新执行上下文。

2 第零章：为什么内核必须先解决“控制权”问题

2.1 受限直接执行：又要快，又不能失控

最简单的高性能方案当然是让用户程序直接在 CPU 上运行：

Program → CPU

但若程序执行：

```
while (1) { }
```

且 OS 没有任何硬件帮助，它可能永远拿不回 CPU。

因此产生核心矛盾：

Direct Execution for Performance

vs.

Restricted Control for Safety

解决它依赖**特权级 + 陷入/返回机制 + 时钟中断**。

2.2 用户态与内核态：不是“两个程序”，而是权限边界

CPU 至少区分两类执行权限：

对比维度	用户态 (User Mode)	内核态 (Kernel Mode)
能否执行特权指令	受限（仅非特权指令）	可以（完整指令集）
能否直接配置页表/设备	通常不允许（引发硬件异常）	内核可以（拥有写 CR3 与 I/O 权限）
运行的代码	普通应用 / 用户函数库	内核代码 / 驱动程序
设计目标	隔离进程、限制非法破坏	虚拟化资源、管理全局安全

关键理解

用户态/内核态是 CPU 执行权限状态，不等于“当前是不是 OS 进程”。同一个用户进程发起系统调用后，可以在同一进程上下文中执行内核代码。

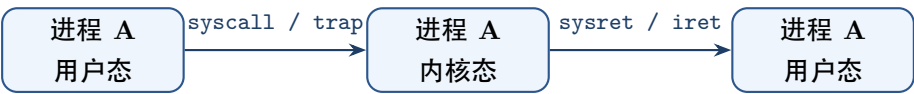
2.3 进入内核的三种典型入口

入口	同步性	典型来源	直观含义与响应动作
系统调用	同步	当前程序显式请求（ <code>trap / syscall</code> ）	“我需要内核提供服务”
异常/陷阱	同步	当前指令执行产生（除零 / 缺页）	“这条指令遇到了特殊硬件情况”
外部中断	异步	定时器、磁盘 I/O、网卡到达	“外部硬件设备发生了事件”

它们都可能使 CPU 从 user mode 进入 kernel mode，但**原因不同、后续处理也不同**。

2.4 Trap 之后不一定 Context Switch

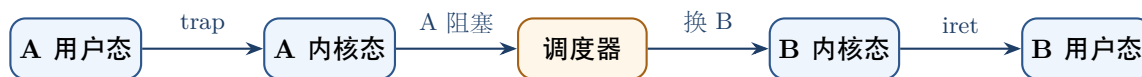
这是本模块第一大易错点。



这只是：

Mode Switch (模式切换)

若 A 在内核中因为等待 I/O 不能继续，才可能出现：



这才包含：

Context Switch

408 必须区分

- 进入内核态 $\neq$ 一定切换进程；
- 发生中断 $\neq$ 一定进行调度；
- 进行调度 $\neq$ 一定切换到不同实体（可能仍选中当前实体）；
- 阻塞通常意味着当前执行实体暂时不能继续，因此会触发调度机会。

3 程序、进程与线程：把“资源世界”和“执行路径”拆开

3.1 程序是静态蓝图，进程是动态执行环境

考试层面可以记：程序是静态的指令和数据集合；进程是程序的一次执行实例。

但更有用的结构是：

Process = Resource/Protection Context + one or more Execution Contexts

一个典型进程包含：

- 虚拟地址空间及其页表/映射；
- 打开的文件描述符与其他内核资源引用；
- 身份、权限、信号等控制信息；
- 一个或多个线程所对应的执行上下文。

3.2 线程是执行上下文，而不是第二套资源世界

线程最稳定的理解是：

Thread = PC + Registers + Stack + SchedulingState + ThreadMetadata

同一进程内线程通常共享：

Code, Global Data, Heap, Address Space, Open Files

而每个线程至少需要独立维护：

PC, Registers, Stack, Execution/Scheduling State

不要把“线程是调度基本单位”绝对化

在 408 的现代内核线程模型中，可以把线程理解为 CPU 调度的基本执行实体；但纯用户级线程若内核不可见，内核实际调度的是其承载的内核调度实体。手册中应把“408 教材表达”和“实现边界”分开写。

3.3 并发与并行

- **Concurrency (并发)**: 多个逻辑执行流在一段时间内都取得进展，不要求同一物理时刻同时执行。
- **Parallelism (并行)**: 多个执行流在不同核心/执行单元上物理同时执行。

不要把并发仅理解成“单核伪同时”。多核系统同样存在并发，只是部分并发执行流可能同时并行。

3.4 408 视角下的进程/线程对比

对比维度	进程 (Process)	同一进程内的线程 (Thread)
核心角色	资源分配与保护环境	独立执行路径与 CPU 调度实体
地址空间	通常彼此独立隔离	共享所属进程的整个虚拟地址空间
切换开销	跨进程切换通常还要更换地址空间上下文，成本较高	同进程线程共享地址空间，通常不需更换地址转换上下文
通信机制	需要 IPC / 共享内存映射	可直接读写共享堆/全局变量（需同步）
故障隔离	隔离性强，崩溃通常不影响其他进程	共享地址空间，致命错误常导致整个进程崩溃

4 PCB：把“进程是什么”落实到内核可管理的数据结构

4.1 PCB 不是进程本体，而是内核管理进程的控制记录

408 中可以把进程控制块 (Process Control Block, PCB) 理解为：

PCB = Identity + Processor State + Scheduling/Control + Resource References

它解决的问题不是“程序的数据放在哪里”，而是：内核怎样识别这个进程、暂停它、重新调度它，并找到它所拥有或引用的资源。

PCB 信息类别	典型字段 / 信息	解决的问题
进程描述信息	PID、父进程关系、用户/权限标识等	“它是谁，属于谁？”
处理机状态信息	PC、SP、PSW/状态寄存器、通用寄存器等需要保存的执行现场	“下次从哪里、以什么 CPU 状态继续？”
调度与控制信息	当前状态、优先级、时间片/调度参数、队列链接、等待原因等	“现在能不能运行，排在哪儿？”
资源与管理信息	地址空间/页表相关引用、打开资源引用、I/O 或其他内核对象引用等	“它依附于哪些资源环境？”

最容易混淆的两层

PCB 保存的是管理信息与资源引用，不等于把进程的代码、堆、栈、文件内容都“装进 PCB”。地址空间、文件对象等由各自子系统维护，PCB/任务结构只持有能找到这些对象的状态或引用。

4.2 PCB 与线程上下文：教材抽象和现代实现要分层

单线程进程的教材模型常把“进程执行现场”一起归入 PCB；引入多线程后，更清楚的拆法是：

Process Resource Context + per-thread Execution Context

每条线程必须有独立 PC、寄存器、栈和调度状态；共享地址空间、文件等进程级资源。工程实现中这些信息可能分散在 task/thread/process 等多个内核结构中，408 做题先守住语义，不要把某个具体内核结构名反过来当定义。

4.3 TCB：把线程自己的执行状态单独拿出来

408 教材常用线程控制块（Thread Control Block, TCB）表示一条线程对应的控制记录。最稳妥的语义模型是：

TCB = Thread Identity + Execution Context + Scheduling State + Thread-local Metadata

判断问题	PCB / 进程级状态	TCB / 线程级状态
“属于谁？”	PID、进程身份与权限、父子关系等	TID、线程身份、所属进程关系等
“资源世界是什么？”	地址空间、打开资源等进程级上下文或其引用	通常不再复制一套独立进程资源世界
“下一条从哪执行？”	单线程教材模型可把执行现场并入 PCB	每线程独立的 PC、SP、寄存器与栈
“现在能否获得 CPU？”	进程级教材模型可记录调度状态	内核可见线程模型下，每线程有自己的 Ready/Running/Blocked 等调度状态

PCB / TCB 的判定口令

先问题目描述的是资源与保护环境，还是一条独立执行路径的现场。前者归到进程级上下文；后者归到线程级上下文。不要靠“字段名长得像 PCB 还是 TCB”猜答案。

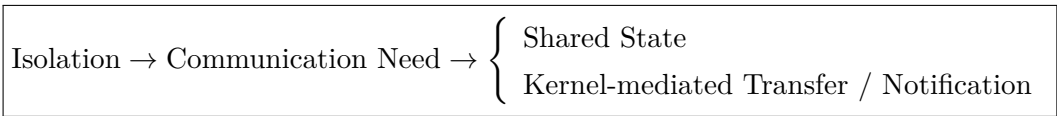
TCB 是教材抽象，不是跨系统统一的结构体名字

不同 OS、线程库和运行时可能把线程状态拆进不同结构。考试中使用 PCB/TCB 术语没有问题；进入工程层后，应追踪“谁保存线程身份、寄存器现场、栈与调度状态”，而不是要求源码中一定存在一个名为 TCB 的独立对象。

5 进程通信 IPC：隔离之后怎样交换信息

5.1 母问题：进程彼此隔离，为什么还能通信？

进程模型首先承诺隔离：一个进程不能默认把另一个进程的普通私有地址直接当作自己的内存读写。现实程序又必须协作，因此 OS 必须提供受控的跨进程信息通道：



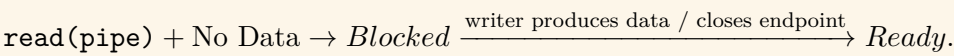
这比背“管道、消息队列、共享内存、信号”更重要：先判断跨进程传递的究竟是共享状态、数据流/消息，还是事件通知，再判断对应对象、队列和阻塞条件。

5.2 四类典型 IPC：看“共享什么、等什么”

IPC 形式	共享/传递的核心对象	典型等待条件	主要机制与代价
共享内存	双方映射到同一共享内存对象/物理实现	数据本身不会自动产生“可读事件”；同步条件由程序协议定义	数据访问路径短，但共享写会产生 race，需要 OS-03 的同步机制；映射细节归 OS-04
管道 (Pipe)	内核维护的有序字节流与缓冲状态	读端遇到“当前无数据但仍可能有后续写入”；写端也可能因容量/语义受限而等待	通过内核对象连接读写端，天然具有 stream、buffer、block/wakeup 语义
消息传递	内核/运行时维护的 message / mailbox / queue	无可接收消息、队列容量不足或同步发送尚未匹配接收方	边界清楚，便于隔离；通常需要排队、复制或引用管理
信号/事件通知	“某事件发生”的控制信息	通常不是大块数据传输通道	信息量小但适合通知、唤醒或控制；具体递送语义依机制而异

IPC 题第一动作：先画对象，再画状态

不要先问“这是哪一种 IPC 名字”。先写：通信对象在哪里？当前有什么状态？哪一方在等待什么条件？哪个事件会改变条件？例如匿名管道的进程侧轨迹可以压成：



注意最后通常只是 **Ready**，并不保证读取者在事件发生瞬间立刻 **Running**。

5.3 共享内存为什么“快”，又为什么不等于“简单”

共享内存把主要数据访问变成普通 load/store，避免每次逻辑通信都必须经过“发送一份消息 → 内核缓冲 → 接收一份消息”的路径。但它只解决**双方怎样看见同一份状态**，并没有自动解决：

- 两个执行流同时修改时谁先谁后；
- 一个线程/进程何时可以认为数据已经准备完成；
- 是否存在 race、lost update、可见性或一致性问题。

因此：

Shared Memory IPC $\neq$ Synchronization

共享映射的建立与地址空间机制由 OS-04 解释；互斥、条件同步与 PV 由 OS-03 解释。本册只拥有“**进程为何需要通信、通信对象如何影响 Blocked/Ready 状态**”这一层。

IPC 跨 Topic 停止规则

若题目问“两个进程怎样交换信息、谁会阻塞、什么事件唤醒”，留在本册；若继续追问共享页如何映射，转 OS-04；追问 mutex/PV/条件变量，转 OS-03；追问 fd/OFD/inode 生命周期，转 OS-06/07；追问 TCP/UDP socket 的运输层端点语义，转 Network。**不要让 IPC 变成复制四个专题正文的借口。**

6 进程资源接口：本册只追踪“引用怎样随生命周期交接”

进程确实需要保存打开文件等资源的引用，但根据本项目的 Topic 边界，本册只追到：

Process → fd table → File-System Open State

后一层的 OFD、offset、dentry/inode 与文件生命周期由文件系统专题完整解释。

这里仅保留与进程生命周期直接相关的三条接口语义：

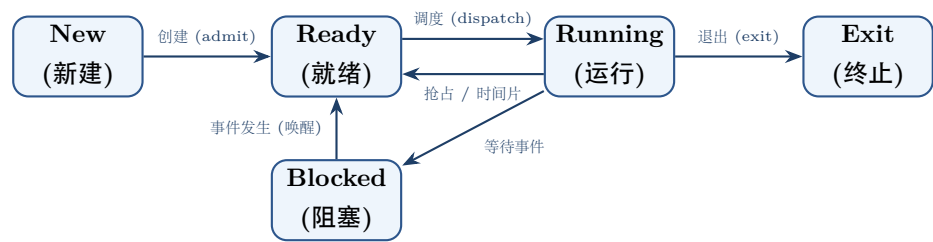
- **fork()**：子进程获得自己的进程管理状态与 fd 表语义副本；对应项可继续引用同一打开实例，因此资源可以跨父子进程共享；
- **exec()**：替换程序映像，不等于创建新进程；未被 close-on-exec 规则关闭的 fd 可以继续存在；
- **exit()**：结束进程执行并释放其持有的运行时引用；“文件对象何时真正消失”属于文件系统生命周期问题。

跨 Topic 时的停止规则

当题目只问进程状态、调度或 **fork/exec**，追到“资源引用被继承/保留/释放”即可；只有题目继续追问 offset、硬链接、inode、unlink 或崩溃一致性时，才进入 OS-06/07 的文件系统模型。

7 进程生命周期主线

7.1 生命周期总览



状态转换的第一原则

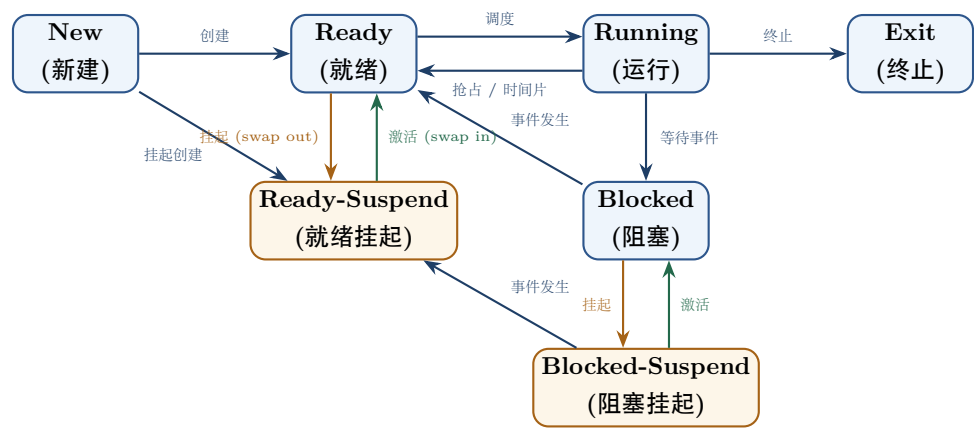
每条状态箭头都必须有事件依据。如果说不出事件，就不应随意改变状态。

7.2 五态为什么还不够：挂起与七态模型

五态模型回答“任务能不能运行、是否正在运行”；当系统还要表达暂时移出内存/暂停参与调度时，需要再增加 Suspend 维度：

Runnable/Blocked 状态 × Resident/Suspended 状态

扩展后的完整七态模型（包含就绪挂起与阻塞挂起）结构如下：



关键转换按“事件是否满足 + 是否重新激活”推：

状态转换	触发事件	转换原因与含义
Ready → Ready-Suspend	suspend / swap out	本来能运行，但暂时被调出内存
Blocked → Blocked-Suspend	suspend / swap out	等待事件的同时被调出内存
Blocked-Suspend → Ready-Suspend	所等事件发生	阻塞原因消失，但尚未激活回内存
Ready-Suspend → Ready	activate / swap in	已满足运行条件且重新调入内存就绪队列
Blocked-Suspend → Blocked	activate (事件未发生)	调回内存后仍然缺少等待条件

Suspend 与 Blocked 不是一回事

Blocked 的判据是“即使现在给 CPU 也无法继续”；Suspend 的判据是“当前被系统暂停/移出正常可调度驻留集合”。因此事件可以让 Blocked-Suspend 先变成 Ready-Suspend，却不会凭空让它直接 Running。

七态模型与后文**中级调度**是同一件事的状态侧与机制侧：状态图告诉你“对象去了哪里”，中级调度告诉你“谁负责做这次挂起/激活”。

7.3 阶段 A：创建——从父进程产生新执行环境

Unix/POSIX 语义中，`fork()` 创建子进程。关键不是“复制所有物理内存”，而是创建一个新的进程语义实体，并让其初始状态看起来接近父进程。

`fork()` 成功后，父子进程都会从“这次调用返回之后”继续执行，但返回值不同：

parent: return child PID	child: return 0
--------------------------	-----------------

因此代码中的 `if (fork()==0)` 本质是在用返回值区分当前这条执行流到底属于父还是子。

连续 fork 题不要机械写 2^n

第一动作是沿控制流逐次问：当前有多少条执行流真正执行到了这一条 `fork`？每一个成功执行到的 `fork()` 才把对应执行流复制成父、子两条。若前面存在 `if/else`、`exit()`、循环或某些分支只有父/子能进入，就不能把源码中出现的 `fork` 个数直接代入 2^n 。

现代实现常通过 COW 优化：

初始共享物理页 + 只读保护 $\xrightarrow{\text{首次写}}$ 复制对应页

文件资源采用另一套语义：父子有各自的 fd 表语义副本，但对应条目可以继续引用同一个文件系统打开实例。这里追踪到“引用关系被继承”为止，打开实例内部状态交给文件系统专题。

7.4 阶段 B：exec——不是新建进程，而是替换进程映像

`exec()` 最应该记住一句：

same process identity, new program image
--

典型结果：

- 进程 PID 不因为 `exec` 本身变成另一个新 PID；
- 原有代码、数据、栈等程序映像被新的可执行程序替换；
- 未设置 `close-on-exec` 的打开文件描述符可以跨 `exec` 保留。

7.5 阶段 C：Ready——只缺 CPU

Ready 的精确定义不是“正在等待某事”，而是：

运行所需条件已满足，只差 CPU 执行机会

因此就绪队列中的实体如果此刻拿到 CPU，就能继续执行。

7.6 阶段 D: Running——用户态和内核态都可能属于 “这个进程正在运行”

当进程执行系统调用进入内核时，常常仍然是当前进程上下文。

因此：

$Running \neq User\ Mode$

进程可以在内核中替自己执行系统调用处理，随后返回用户态；也可以在内核中发现条件不满足并进入阻塞。

7.7 阶段 E: Blocked——即使给 CPU 也不能继续

Blocked 与 Ready 的核心区别：

对比维度	就绪态 (Ready)	阻塞态 (Blocked)
缺少的核心资源	只缺 CPU 时间片	缺少特定的外部事件或资源条件
给 CPU 能否继续运行	能立即投入 Running 运行	不能，即使分配 CPU 也无法向前推进
进入触发原因	被抢占、刚创建完成、从 Blocked 被唤醒	等待磁盘 I/O、信号量 P 操作、子进程退出等
离开触发原因	调度器选择 (Scheduler Pick)	所等待的外部事件/资源就绪

“调用 read 就 Running→Blocked” 也是过度简化

read() 如果数据已经在缓存/缓冲中，可能直接完成并返回。只有在当前语义要求等待且数据尚未可用时，执行实体才可能睡眠/阻塞。类似地，用户态 mutex 的无竞争获取甚至可能不需要系统调用。

7.8 阶段 F: Wakeup——Blocked → Ready，不等于马上 Running

设备完成、中断到来、条件被满足后，内核通常只是把等待实体变成 runnable/ready：

$Blocked \rightarrow Ready$

它仍然要等待 scheduler 选择：

$Ready \rightarrow Running$

这是 408 状态题最关键的两步分离。

7.9 阶段 G: Exit / Zombie / Wait

进程执行 exit() 后，绝大多数运行资源会释放，但 Unix 风格系统通常保留少量退出状态，供父进程通过 wait() 回收，因此出现 zombie 概念。

应理解为：

$Process\ Execution\ Ends \neq All\ Kernel\ Metadata\ Immediately\ Vanishes$

8 调度：控制权已经回到内核，下一步给谁？

8.1 先分清三级调度：它们选择的不是同一种“下一位”

“调度”不是只有 ready queue 里挑一个进程。408 经典模型按决策层次分为：

层次	又称	典型状态/位置变化	决策问题
高级调度	作业调度	后备作业 → 被接纳进入系统/内存并建立进程	“哪些作业现在可以进入执行系统？”
中级调度	内存调度 / 对换相关调度	Ready/Blocked ↔ Ready-Suspend/Blocked-Suspend	“哪些进程暂时挂起，哪些重新激活？”
低级调度	进程调度 / CPU 调度	Ready → Running	“当前 CPU 下一步运行谁？”

一眼定位

看到“后备队列/作业接纳”先想**高级调度**；看到“挂起、激活、对换”先想**中级调度**；看到“就绪队列、时间片、CPU 分配”先想**低级调度**。本册后续 FCFS/SJF/RR 等算法主要讨论低级 CPU 调度。

教材模型与现代系统边界

三级调度是 408 的经典抽象。现代通用操作系统未必存在传统批处理意义上的“作业调度”，挂起/换出机制也不会机械等同于整进程 swap。考试先按题设和教材模型作答，再区分工程实现。

8.2 机制与策略必须拆开

机制 / 策略

机制 (mechanism)：OS 有没有能力停止当前执行流、保存状态、切换到另一个执行流？
策略 (policy)：面对多个 runnable 实体，应该选择谁？

例如：

Timer Interrupt + Context Switch Machinery = Mechanism

而：

FCFS/SJF/RR/Priority/MLFQ = Policy

8.3 调度评价指标是互相冲突的目标函数

- CPU utilization：尽量别让 CPU 闲；
- Throughput：单位时间完成更多工作；
- Turnaround time：提交到完成；
- Waiting time：在 ready queue 中等待的时间；
- Response time：从请求到首次响应；
- Fairness：避免长期得不到 CPU；

- Predictability / deadline：实时系统特别关注。

不存在对所有工作负载同时最优的单一调度策略。

8.4 先识别工作负载，再决定“什么叫好”

调度题不应从算法名字开始，而应先确定系统最想保护的目标：

Workload / System Goal → Objective → Scheduling Policy

典型系统/工作负载	首要关注目标	看到题目时的第一判断
批处理 (Batch)	Throughput、Turnaround，尽量高效完成一批作业	更关心单位时间完成多少、平均多久完成，而不是每个任务立刻得到一次响应
分时/交互式 (Time-sharing / Interactive)	Response、Fairness，同时控制切换开销	用户不能长时间感觉“系统没理我”，因此要让 runnable 任务周期性获得执行机会
实时 (Real-time)	Deadline、Predictability / bounded latency	“平均很快”不等于合格；关键是关键任务能否在规定时间内边界内完成或响应

调度算法选择题的第一动作

先圈题干中的吞吐/周转、首次响应/交互、公平、截止时间等目标词，再判断算法的偏好是否与目标一致。不要看到“现代系统”就条件反射选 RR/MLFQ，也不要看到“平均等待时间”就忽略题目给出的抢占条件与 burst 信息。

8.5 经典算法应按“偏好谁”来记

算法	核心偏好	主要优点	核心代价 / 潜在风险
FCFS	先到先服务	实现简单、低调度系统开销	Convoy Effect（护航效应），短任务久等
SJF	短 CPU Burst 优先	理论上可获得最小平均等待时间	难准确预测下一步 Burst，长任务易饥饿
SRTF	剩余时间短者优先	抢占式偏向短任务，降低响应时间	频繁抢占与上下文切换开销大
HRRN	当前响应比最高者优先	同时考虑等待时间与服务时间，长任务等待越久竞争力越强	非抢占式；仍需知道/估计服务时间，计算开销高于 FCFS
Priority	优先级高者优先	直观表达任务重要程度	低优先级任务长时期饥饿（需 Aging 机制）
RR	人人轮流切时间片	响应极佳、保证公平性	时间片 q 过小导致频繁切换；过大退化为 FCFS
MLFQ	用历史行为推断类型	兼顾短交互任务与长计算任务	规则复杂，需要防范 Task Gaming 与饥饿

8.6 HRRN：不要只背公式，要看“等待如何改变优先级”

高响应比优先（Highest Response Ratio Next, HRRN）在每次**非抢占式**选择时计算：

$$R = \frac{W + S}{S} = 1 + \frac{W}{S}$$

其中 W 是已经等待的时间， S 是要求服务时间。选择当前 R 最大的作业/进程。

这个式子可以直接从两个极端理解：

- 若等待时间相近， S 小的短任务响应比更高，保留了 SJF 偏爱短任务的倾向；
- 同一任务等待越久， W 持续增大，响应比随之上升，因此经典模型下能显著缓解长任务饥饿。

HRRN 计算题第一动作

不要在所有时刻连续重算。只在 CPU 需要重新选择下一个任务的决策点，对当时已经到达且尚未完成的候选者计算 W 与 R ；一旦选中，因为 HRRN 是非抢占式，该任务运行到本次服务结束。

教材模型 vs 现代 Linux 调度算法

MLFQ 是 408 考纲常考的经典教学策略；现代 Linux 普通任务调度长期采用 CFS 的“公平 CPU”模型，当前内核正在向 EEVDF 方向演进。备考时需以经典 MLFQ 规则为准，理解现代算法对公平性的优化。

8.7 时间片到底解决什么

时间片不是越小越公平就越好：

$$q \downarrow \Rightarrow \text{Response may improve, } \text{ContextSwitchOverhead} \uparrow$$

$$q \uparrow \Rightarrow \text{Overhead} \downarrow, \quad RR \rightarrow FCFS$$

因此调度本质上是：

$$\text{Latency} \leftrightarrow \text{Throughput} \leftrightarrow \text{Fairness} \leftrightarrow \text{Overhead}$$

8.8 有调度原因，不等于“现在这一条指令就能切换”

408 选择题常把三个概念混成一个：

$$\text{出现调度原因} \neq \text{允许立即调度} \neq \text{最终一定换人}$$

某个事件可以先产生 reschedule 的需求，但真正的上下文切换必须发生在内核允许切换的安全点。

经典考试语境下，以下阶段通常不能把当前进程随意切走：

1. **正在处理中断/异常的关键现场**：硬件与内核首先必须把当前处理路径维护到可安全返回或可调度的位置；定时器中断可以提出抢占需求，但不等于在中断处理任意一条指令处直接 context switch；
2. **正在执行不可抢占的内核临界区**：如果切走会暴露半更新的内核共享状态，就必须先离开该临界区；

3. **正在执行必须保持原子性的原语/关键原子操作**：既然语义要求不可分割，中途自然不能通过普通调度把它拆开。

“处于临界区就绝不能调度”是错误绝对化

这里说的是**不可抢占的内核临界区/原子区间**。普通用户程序进入自己的临界区，并不会因此自动获得“CPU 不可被抢占”的特权；用户临界区的互斥正确性必须自己由同步机制保证。

调度安全点模型

做题时分两步判断：先问“**是否已经出现调度机会/调度请求?**”，再问“**当前是否处在允许 context switch 的安全点?**”。只有第二问也成立，才继续判断调度器最终是否选择另一个执行实体。

9 上下文切换：到底切了什么？

9.1 抽象定义

上下文切换至少包括：

1. 保存旧执行实体下一次继续所需的 CPU 执行状态；
2. 更新其调度/运行状态；
3. 选择新执行实体；
4. 恢复新实体的 CPU 状态；
5. 必要时切换地址空间/内存上下文；
6. 从新实体的正确位置继续执行。

9.2 同进程线程切换与跨进程切换

同进程线程共享地址空间，因此通常无需更换整个地址转换上下文；跨进程切换往往还需要切换地址空间相关状态。

考点辨析：上下文切换与 Cache/TLB 刷新

408 经典模型常把地址空间切换视为 TLB 需要刷新或重新建立有效翻译；但在现代 CPU 工程实现中，可通过 ASID / PCID 标签保留部分 TLB 项，不必简单一刀切清空。

9.3 动态轨迹：A 切到 B

比“保存 PCB → 恢复 PCB”更好的理解是：

$$A_{user} \rightarrow A_{kernel} \rightarrow Scheduler \rightarrow B_{kernel} \rightarrow B_{user}$$

其中 scheduler 本身运行在内核控制路径中。

10 系统调用、中断、异常：控制流为什么会改变

10.1 中断响应要拆成两层：硬件自动入口与软件处理程序

408 常把“CPU 响应中断”压成一句话，容易让学生把所有现场保存都误以为是硬件自动完成。更稳定的分层是：

Hardware Entry → Software Handler → Return/Schedule

1. **硬件自动入口（中断隐指令的教材抽象）**：CPU 在满足响应条件时建立一个可返回的受保护入口。
408 经典表述通常概括为**关中断/暂时屏蔽同级可屏蔽中断、保存断点与必要状态、根据中断向量引出处理程序**。抽象上至少要保证返回所需的 PC/状态信息不会丢失，并切入内核控制路径；
2. **软件入口保存现场**：中断处理程序的入口代码继续保存需要保护的通用寄存器、必要的控制状态/屏蔽信息等。“保存断点”与“保存完整现场”不是同一个动作；
3. **执行中断服务**：确认中断源、处理设备/时钟事件、更新内核状态，必要时把等待任务从 Blocked 改为 Ready。若体系结构与内核策略允许中断嵌套，可以在建立好安全现场后重新允许更高优先级/合适类别的中断；
4. **恢复与返回**：恢复软件保存的现场，再通过体系结构规定的中断返回指令恢复硬件入口保存的返回状态。若这次中断产生了抢占需求，也应在允许调度的安全点先进入 scheduler，而不是在处理程序任意位置强行切走。

408 高频辨析

- 硬件保存**最小返回状态**，不等于硬件替软件保存了所有通用寄存器；
- “中断到来”只意味着获得一次内核控制机会，不等于一定调度，更不等于一定换进程；
- 经典“关中断 → 保存断点 → 引出 ISR”是教材级抽象；不同体系结构对 PC/PSW、栈切换、屏蔽位的具体自动动作可以不同。

中断、异常与系统调用不要机械套同一条微观顺序

三者都可能进入受保护的内核入口，但来源与恢复语义不同：外部中断是异步事件，异常由当前指令触发，系统调用是程序主动请求。考试若明确问“中断隐指令”，按中断响应模型作答；若问缺页异常或 syscall，则沿各自的 trap/exception 语义推演。

10.2 一次系统调用的五阶段模板

1. 用户代码准备参数；
2. 执行体系结构规定的 trap/syscall 指令；
3. 硬件保存足够的返回状态并进入内核入口；
4. 内核验证参数、执行服务；
5. 若可立即完成则 return；若必须等待，则阻塞当前实体并调度其他实体。

因此系统调用有两条最重要分支：

Fast: syscall $\rightarrow$ return same task

Slow: syscall $\rightarrow$ block $\rightarrow$ schedule someone else

10.3 时钟中断为什么是抢占式调度的基础

若用户程序不主动进入内核，定时器仍能周期性打断 CPU：

User $\rightarrow$ TimerInterrupt $\rightarrow$ Kernel

这使 OS 有机会：

- 记账；
- 更新运行时间；
- 判断时间片/优先级；
- 必要时触发抢占和重新调度。

10.4 设备中断：唤醒并不等于直接交 CPU

典型 I/O 完成：

Device $\rightarrow$ Interrupt $\rightarrow$ KernelHandler $\rightarrow$ Wakeup(Task)

关键状态变化通常只是：

Blocked $\rightarrow$ Ready

之后是否立即抢占当前任务，取决于调度策略和系统状态。

11 综合题一：从 fork 到 wait 的进程生命周期

推荐课堂主线

不要先给学生五态图。先让学生跟踪一个真实进程：shell fork $\rightarrow$ child exec $\rightarrow$ schedule $\rightarrow$ syscall $\rightarrow$ block $\rightarrow$ wakeup $\rightarrow$ exit $\rightarrow$ parent wait。五态图应当从这个动态故事中“长出来”。

11.1 完整轨迹

1. Shell 正在 Running。
2. Shell 调用 fork() 进入内核。
3. 内核创建 child 的管理状态、地址空间语义（常用 COW）和继承资源引用。
4. child 进入 Ready；parent 也可能继续 Ready/Running，谁先运行不由 fork() 本身保证。
5. child 被调度后运行，调用 exec()。

6. `exec()` 替换 child 的程序映像，但保持该进程身份；未 `close-on-exec` 的 fd 可继续存在。
7. 新程序运行并发起一个系统调用。
8. 若系统调用可立即完成：kernel $\rightarrow$ user，仍是同一进程。
9. 若必须等待 I/O：Running $\rightarrow$ Blocked，进入等待队列，调度器选择另一个 Ready 实体。
10. 设备完成，中断进入内核；等待条件满足后 child：Blocked $\rightarrow$ Ready。
11. child 某次再次被调度：Ready $\rightarrow$ Running。
12. child 最终 `exit()`，释放大运行资源，留下退出状态成为 zombie (Unix 语义)。
13. parent 的 `wait()` 获取退出信息并完成最终回收。

12 综合题二：一次阻塞 read 的“进程侧投影”

假设进程 A 调用：

```
read(fd, buf, 4096)
```

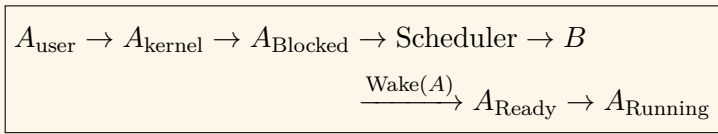
且底层条件使本次调用必须等待。这个专题只跟踪**控制权与 task state**：

1. A 在用户态 Running。
2. syscall 使 CPU 进入 A 的内核上下文；**此刻仍然是 A 在运行，不是已经切换进程。**
3. 内核沿文件/I/O 子系统处理请求。若请求能立即完成，A 直接沿内核返回路径回到用户态，不发生 Blocked。
4. 本题设定必须等待：A 暂时不能继续，故 Running $\rightarrow$ Blocked，并挂到相应等待结构；
5. CPU 已经不能继续执行 A 的工作，scheduler 从 Ready 集合选择其他执行实体，必要时发生 context switch；
6. 后续底层事件完成后，相关子系统调用 wakeup，使 A：Blocked $\rightarrow$ Ready；
7. **wakeup 不等于立刻获得 CPU。**A 仍要等待低级调度；
8. A 再次被选中：Ready $\rightarrow$ Running，从原等待后的内核控制路径继续，最终完成 syscall 并返回用户态。

为什么这里故意不展开 fd/OFD/DMA/Page Cache

这些对象分别由文件系统与 I/O 专题拥有。OS-01/02 只需要知道：**某个外部条件未满足会使当前 task block；条件完成会使它 wake 到 Ready；何时再次 Running 由调度决定。**若题目要求完整追踪 `read()` 的文件对象、缓存、块映射、DMA 与完成中断，应切换到 OS 综合专题，把各 Topic 的局部模型串起来。

本专题真正要拿下的链



这条链同时检查 mode switch、block、wakeup、ready queue、调度机会与 context switch，但不把别的子系统细节重复背一遍。

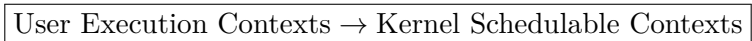
13 线程实现模型：考试模型与现代实现边界

13.1 用户级线程与内核级线程的区别：谁看得见、谁调度

对比维度	用户级线程 (ULT)	内核级线程 (KLT)
调度掌控者	用户态线程库 / 运行时 (Runtime)	OS 内核调度器
内核视角透明度	内核完全不可见（仅看到承载进程）	内核直接感知并维护每条线程状态
切换是否进内核	无须内核参与，纯用户态上下文切换	必须经过中断/Trap 进入内核态完成切换
多核利用能力	M:1 模型无法利用多核并行执行	可直接在多核 CPU 上同时调度并行执行
阻塞调用的影响	在纯 M:1 模型中，一个 ULT 的阻塞系统调用可能阻塞整个承载进程	一个 KLT 阻塞通常不妨碍同进程其他 KLT 运行

13.2 M:1、1:1、M:N 的本质

它们讨论的是：



如何映射。

现代 Linux POSIX 线程通常采用 1:1 内核线程模型；Go goroutine、某些语言虚拟线程等则在语言运行时层面重新引入 M:N 式用户级调度思想。

14 408 解题检查表：从概念辨析到状态推演

14.1 进程状态题八问

- 1. 当前实体属于 New、Ready、Running、Blocked、Exit 中哪一类基本状态？
- 2. 题目是否额外引入 Suspend？若引入，它现在是 Ready-Suspend 还是 Blocked-Suspend？
- 3. 它当前缺的是 CPU、外部事件，还是“重新激活/换入”条件？
- 4. 发生了什么事件？这条状态箭头是否有明确事件依据？
- 5. 若等待事件发生，Blocked-Suspend 应先到 Ready-Suspend 还是可以直接 Running？

6. 这次变化涉及高级、中级还是低级调度?
7. 事件只是改变 task state, 还是同时产生调度机会?
8. 是否混淆“进程在内核态执行”和“已经换成另一个进程”?

14.2 调度计算题九步

1. 先判断题目讨论高级/中级/低级调度中的哪一层; 经典算法计算题通常是低级 CPU 调度;
2. 圈出题目真正优化的目标: throughput/turnaround、response/fairness, 还是 deadline/predictability;
3. 写出 arrival time、burst/service time、priority、quantum 等输入;
4. 确定算法是抢占还是非抢占;
5. 每个真正的决策事件点重新确定当前 ready set;
6. 若是 HRRN, 只在需要重新选任务时计算 $R = 1 + W/S$;
7. 按策略选择, 并明确同值时题目采用什么 tie-break;
8. 画 Gantt 图并逐段检查到达、完成、抢占事件;
9. 从完成时刻反推 turnaround / waiting / response, 最后检查单位与定义。

14.3 系统调用/中断题九问

1. 入口是 syscall、exception 还是 interrupt?
2. 是同步事件还是异步事件?
3. 若题目问中断响应, 哪些是硬件自动入口动作, 哪些由软件 ISR 保存/恢复?
4. CPU 是否发生 privilege/mode transition?
5. 当前是否仍处于原进程的执行上下文?
6. 当前任务处理完事件后还能继续吗?
7. 若不能, 它进入哪个等待队列/状态?
8. 即使产生了调度请求, 当前是否已经到允许 context switch 的安全点?
9. 最终是 return same task, 还是 scheduler 选择 another task?

14.4 PCB / TCB / 进程资源接口题八问

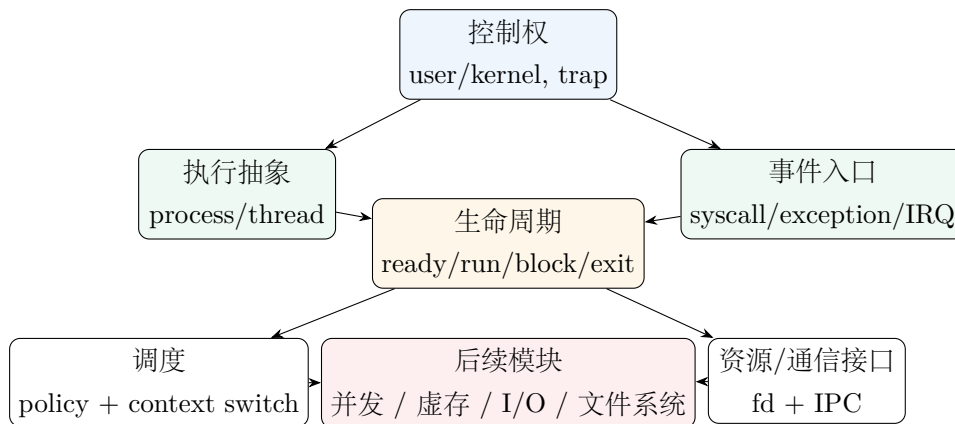
1. 题目问的是进程身份、CPU 执行现场、调度状态, 还是资源引用? 先把 PCB 信息类别定位清楚;
2. 当前信息属于整个进程的 resource/protection context, 还是某一条线程的 execution context? 若是后者, 优先按 TCB 语义定位;
3. 需要恢复执行的是 PC/寄存器/栈等 execution context, 还是进程级 resource context?
4. PCB/任务结构保存的是对象本体, 还是能找到对象的引用与管理信息?

5. `fork()` 后哪些进程管理状态新建，哪些资源引用按语义继承？
6. 若代码根据 `fork()` 返回值分支，父进程得到 child PID、子进程得到 0 后，各自会走哪条控制流？
7. `exec()` 是创建新进程，还是在同一进程身份下替换程序映像？
8. 如果题目继续追问 offset、OFD、inode、unlink 等文件内部语义，立即切换到 OS-06/07，不在本专题继续猜。

14.5 IPC 题六问

1. 两个进程之间要传递的是共享状态、数据流/消息，还是事件通知？
2. 承载通信的对象在哪里：共享映射、pipe buffer、message queue/mailbox，还是 notification state？
3. 当前哪一方缺少什么条件，因此可能 Blocked？
4. 哪个事件改变了通信对象状态，并使 waiter 从 Blocked 变成 Ready？
5. 如果是共享内存，题目是否已经从“能通信”进入了“怎样同步”？若是，切到 OS-03；
6. 如果继续追问映射、OFD/inode 或 transport socket 内部语义，是否应该切到 OS-04、OS-06/07 或 Network？

15 知识树与专题接口



核心结论

这一模块学完，学生看到“进程”时不应只想到 PCB；而应自动看到：

Resource Context + Execution Context + Kernel State

看到“状态变化”时应自动问：

什么事件？谁获得控制？当前还能继续吗？

看到“调度”时应自动先分层、再区分机制与策略：

High / Medium / Low Scheduling

Mechanism $\neq$ Policy

看到“中断到来”时应自动拆开：

Hardware Entry $\rightarrow$ Software Handler $\rightarrow$ Safe Return/Schedule Point

看到“进程通信”时应自动先问：

共享什么/传什么？ $\rightarrow$ 谁在等什么？ $\rightarrow$ 哪个事件改变状态？

看到 `read/fork/exec` 时只完整推演本 Topic 拥有的**控制权**、**task state**、**IPC 进程侧状态与资源引用交接**；进入同步协议、共享页映射、OFD、DMA 或运输层 socket 内部机制时，切换到对应 Topic，而不是在一本手册里重复所有子系统。

参考与工程边界来源

- Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*: 进程、受限直接执行、调度与并发的教学框架。
- MIT PDOS, *xv6: a simple, Unix-like teaching operating system*: trap、scheduler、sleep/wakeup、process lifecycle 的实现级案例。
- POSIX / The Open Group: `fork()`、`exec()` 与进程资源继承/程序映像替换的规范语义。
- Linux Kernel Documentation: task、trap/interrupt、scheduler 以及 CFS/EEVDF 等工程实现边界。